

2026 EDITION

15 Interview Questions

LLM Fundamentals & Architecture

Clear answers • Worked examples • Visual diagrams

Tokens

Embeddings

Attention

RoPE

KV Cache

MoE

RLHF / DPO

Scaling Laws

WHAT'S INSIDE

01	What is an LLM and how does it generate text?	FOUNDATIONS
02	Tokenization and subword BPE	FOUNDATIONS
03	Embeddings and vector meaning	FOUNDATIONS
04	Transformers vs RNNs / LSTMs	ATTENTION MECHANICS
05	Self-attention: Query, Key, Value	ATTENTION MECHANICS
06	Multi-head attention (and GQA)	ATTENTION MECHANICS
07	Positional encoding and RoPE	ATTENTION MECHANICS
08	Anatomy of a Transformer block	ARCHITECTURE
09	Encoder-only vs decoder-only vs enc-dec	ARCHITECTURE
10	Causal masking and autoregression	ARCHITECTURE
11	Decoding: greedy, temperature, top-k, top-p	GENERATION & TRAINING
12	Context window and KV cache	GENERATION & TRAINING
13	Training pipeline: pre-train, SFT, RLHF/DPO	GENERATION & TRAINING
14	Scaling laws and Chinchilla	SCALING & EFFICIENCY
15	Mixture of Experts (MoE)	SCALING & EFFICIENCY

QUESTION 01 / 15

What is a Large Language Model (LLM), and how does it generate text?

SHORT ANSWER

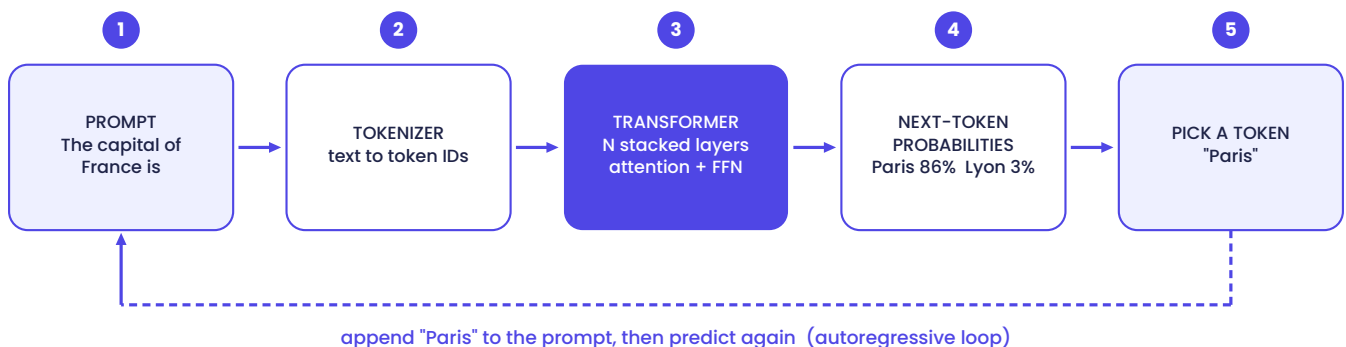
An LLM is a neural network, almost always a **Transformer**, trained on huge text corpora to predict the **next token** given the tokens before it. Generating text is simply that one prediction repeated, one token at a time.

EXPLAINED

- The input text is split into **tokens**, converted to vectors, and processed by many stacked Transformer layers.
- The last layer outputs a **probability distribution** over the whole vocabulary (tens of thousands to 200k+ tokens).
- One token is chosen (greedily or by sampling), appended to the prompt, and the loop repeats until a stop token or length limit.
- "Large" means billions of parameters and trillions of training tokens; abilities such as in-context learning emerge with scale.

DIAGRAM

The next-token prediction loop

**WORKED EXAMPLE**

Prompt: **"The capital of France is"**. The model assigns Paris 86%, Lyon 3%, Rome 2% ... and picks "Paris". The new prompt "The capital of France is Paris" is fed back in, and the model now predicts "." and stops.

INTERVIEW TIP

Say "predicts the next token" and "probability distribution", then add that fluent output does not guarantee factual accuracy.

LIKELY FOLLOW-UP

Why do LLMs hallucinate? They optimise for plausible next tokens, not verified truth; grounding with retrieval (RAG) and citations reduces it.

QUESTION 02 / 15

What is tokenization, and why do LLMs use subword tokens like BPE?

SHORT ANSWER

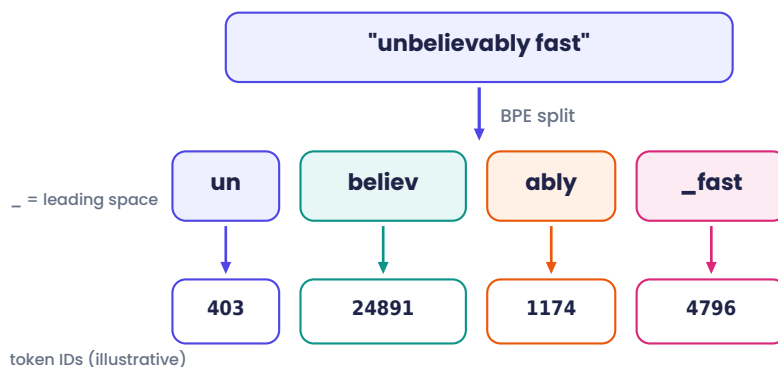
Tokenization converts raw text into integer IDs from a fixed vocabulary. Subword methods such as **Byte-Pair Encoding (BPE)** split rare words into reusable pieces, so the vocabulary stays small and the model never meets an "unknown word".

EXPLAINED

- **Character-level:** tiny vocabulary but very long sequences. **Word-level:** short sequences but a huge vocabulary and unseen words. **Subword** is the sweet spot.
- BPE starts from bytes or characters and repeatedly merges the most frequent adjacent pair until the target vocabulary size is reached.
- Cost, latency and context-window usage are measured in **tokens**, not words. English averages roughly 0.75 words per token.
- Pitfalls: numbers, code and non-English text can split into many tokens, which hurts arithmetic, spelling and cost.

DIAGRAM

Text to subword tokens to IDs

**WHY SUBWORDS?**

- No unknown words
- Vocab of 32k - 200k+ tokens
- Short sequences
- Rare words become pieces
- Common words stay whole
- Works for code & emoji

WORKED EXAMPLE

"unbelievably fast" becomes [un] [believ] [ably] [_fast] and then the IDs [403, 24891, 1174, 4796] (IDs are illustrative). The same sentence in a language with less training data can need noticeably more tokens.

INTERVIEW TIP

Mention that API pricing and context limits are token-based, and that every model family ships its own tokenizer.

LIKELY FOLLOW-UP

Why do LLMs struggle to count letters in a word? They see subword chunks, not characters, so a word such as "strawberry" arrives as a few pieces rather than ten letters.

QUESTION 03 / 15

What are embeddings, and why do they matter?

SHORT ANSWER

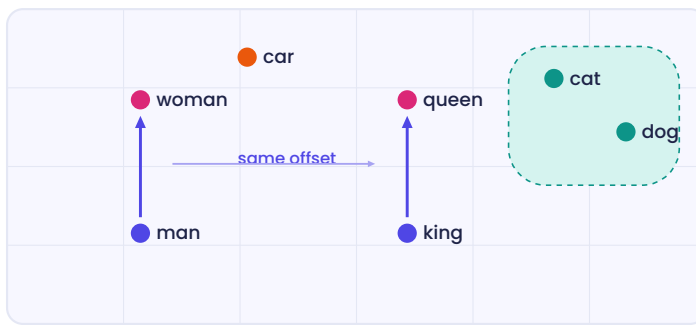
An embedding is a learned, dense vector (for example 4,096 numbers) that represents a token. Tokens with similar meaning end up **close together** in this vector space, so the model can do arithmetic on meaning.

EXPLAINED

- The embedding matrix has shape **vocab_size x d_model**; a token ID simply selects one row.
- Similarity is measured with **cosine similarity** or the dot product of two vectors.
- Directions encode relations: the classic demo is **king - man + woman = queen**.
- Inside an LLM, vectors become **contextual** layer by layer: "bank" in "river bank" and "bank loan" ends up different.
- Many models tie the input embedding and output (unembedding) matrices to save parameters.

DIAGRAM

Meaning as geometry



2-D sketch: real vectors have thousands of dimensions

TOKEN TO VECTOR

cat → [0.21, -0.54, 0.88, ...]

d_model = 4096 numbers per token

COSINE SIMILARITY (illustrative)cat vs dog 0.76cat vs car 0.12**WORKED EXAMPLE**

A 50,000-token vocabulary with d_model = 4,096 gives **50,000 x 4,096 = about 205M parameters** in the embedding table alone. The same idea powers semantic search: embed documents and the query, then return the nearest vectors.

INTERVIEW TIP

Connect embeddings to RAG and vector databases; it shows you can link theory to real products.

LIKELY FOLLOW-UP

How do embeddings differ from one-hot vectors? One-hot vectors are huge, sparse and all equally distant; embeddings are small, dense and encode similarity.

QUESTION 04 / 15

Why did Transformers replace RNNs and LSTMs?

SHORT ANSWER

RNNs read tokens one by one, so training cannot be parallelised and information about early tokens fades. Transformers use **attention** so every token can look directly at every other token, which allows massive GPU parallelism and better long-range modelling.

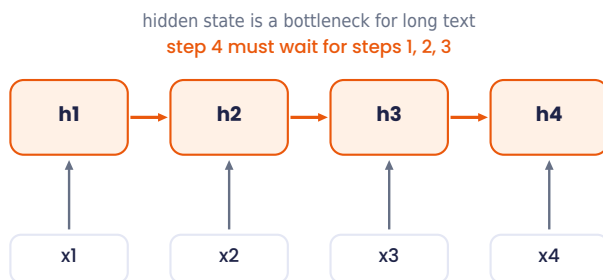
EXPLAINED

- **RNN / LSTM:** the hidden state passes through every step, so the path between distant tokens is $O(n)$ and gradients can vanish or explode.
- **Transformer:** any two tokens are connected by a single attention hop, and all positions are computed simultaneously during training.
- **Trade-off:** self-attention cost grows as $O(n^2)$ with sequence length, which drives research on efficient attention (FlashAttention, sliding windows, GQA).
- Introduced in "Attention Is All You Need" (Vaswani et al., 2017).

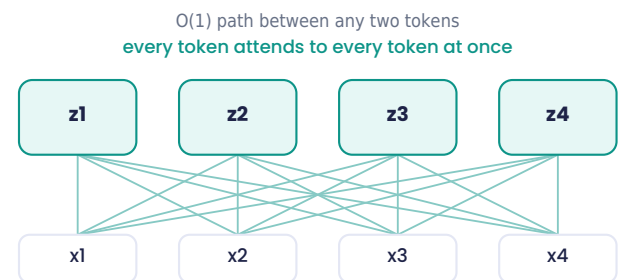
DIAGRAM

Sequential vs parallel processing

RNN / LSTM - sequential



TRANSFORMER - parallel



WORKED EXAMPLE

For a **1,000-token** text an RNN needs 1,000 sequential steps. A Transformer handles all 1,000 positions in one parallel pass per layer, at the price of comparing $1,000 \times 1,000 = 1\text{M}$ token pairs.

INTERVIEW TIP

Name the $O(n^2)$ attention cost as the honest trade-off; balanced answers impress interviewers.

LIKELY FOLLOW-UP

Can a Transformer handle unlimited length? No. The context window is bounded by $O(n^2)$ attention compute and KV-cache memory, which is why long-context tricks exist.

QUESTION 05 / 15

Explain self-attention using Query, Key and Value.

SHORT ANSWER

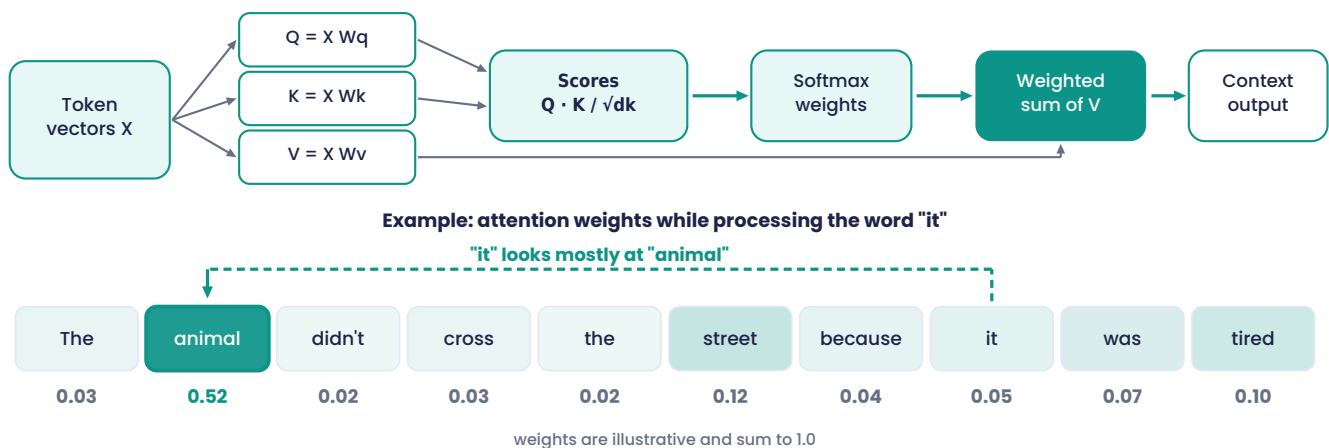
Each token creates a **Query** (what am I looking for?), a **Key** (what do I contain?) and a **Value** (what do I pass on?). Matching queries against keys gives attention weights, which blend the values into a context-aware output.

EXPLAINED

- $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$
- Q, K and V come from multiplying token vectors by learned matrices W_Q, W_K, W_V .
- The dot product $Q \cdot K$ measures relevance; dividing by $\sqrt{d_k}$ keeps scores in a range where softmax gradients stay healthy.
- Softmax turns scores into weights that sum to 1; the output is the **weighted sum of Values**.

DIAGRAM

Self-attention data flow and a weight example



WORKED EXAMPLE

In "The animal didn't cross the street because it was tired", the word "it" puts its highest attention weight (0.52) on "animal", so the model resolves the pronoun correctly.

INTERVIEW TIP

Be ready to explain why we divide by $\sqrt{d_k}$ and what the softmax output represents.

LIKELY FOLLOW-UP

What is cross-attention? The same formula, but Queries come from one sequence (the decoder) while Keys and Values come from another (the encoder output).

QUESTION 06 / 15

What is multi-head attention, and why use several heads?

SHORT ANSWER

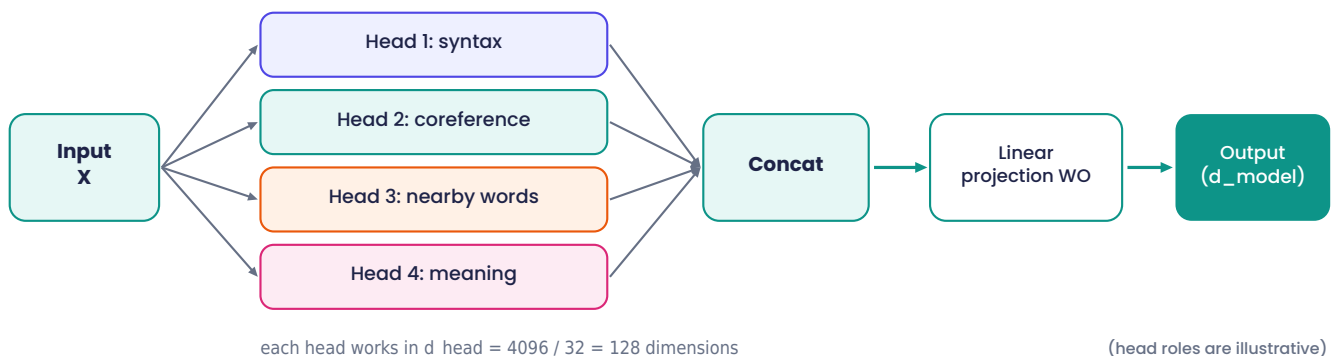
Instead of one large attention operation, the model runs **h smaller attentions (heads) in parallel** on different learned projections, then concatenates and mixes the results. Each head can specialise in a different kind of relationship.

EXPLAINED

- With $d_{\text{model}} = 4096$ and 32 heads, each head works in **128 dimensions**, so total compute stays close to single-head attention.
- Heads tend to learn different patterns: syntax, coreference, nearby-token focus, copying (induction) patterns. Not all are cleanly interpretable and some are redundant.
- Head outputs are concatenated and projected by W_O back to d_{model} .
- Modern variants **MQA** and **GQA** let several query heads share one set of K/V heads, shrinking the KV cache.

DIAGRAM

Heads run in parallel, then merge

**WORKED EXAMPLE**

In "**The chef tasted the soup and added salt**": one head links "added" to its subject "chef", another links "salt" to "soup", and a third just attends to the previous word.

INTERVIEW TIP

Mention GQA; it shows you know production models, not only the 2017 paper.

LIKELY FOLLOW-UP

MHA vs MQA vs GQA? MHA gives every head its own K/V; MQA shares one K/V across all heads; GQA shares K/V within groups of heads, a middle ground.

QUESTION 07 / 15

Why do Transformers need positional encoding? What is RoPE?

SHORT ANSWER

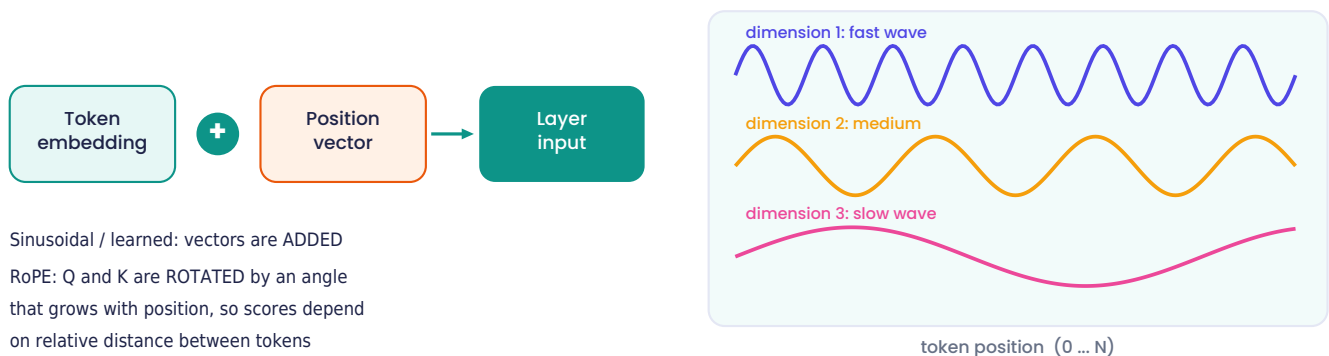
Self-attention is **order-blind**: without position information, "dog bites man" and "man bites dog" look identical. Positional encodings inject word order into the model.

EXPLAINED

- **Sinusoidal** (original 2017): fixed sin/cos waves of different frequencies added to the embeddings.
- **Learned absolute**: a trainable vector per position (e.g. GPT-2); limited to the trained length.
- **RoPE** (Rotary Position Embedding): rotates Q and K by an angle proportional to position, so scores depend on **relative distance**. Used by Llama-family and many modern LLMs.
- **ALiBi** adds a distance penalty to attention scores. Long-context extension methods (position interpolation, YaRN) rescale RoPE frequencies.

DIAGRAM

Adding position information



WORKED EXAMPLE

Without positions, "dog bites man" and "man bites dog" are the same **bag of tokens** {dog, bites, man}. Adding a different position vector to each slot makes the two sentences distinguishable.

INTERVIEW TIP

Contrast absolute vs relative positions and name RoPE as the modern default.

LIKELY FOLLOW-UP

What happens beyond the trained context length? Quality can degrade because those positions were never seen; frequency scaling (YaRN, interpolation) plus brief long-context fine-tuning helps.

QUESTION 08 / 15

Describe the structure of a single Transformer block.

SHORT ANSWER

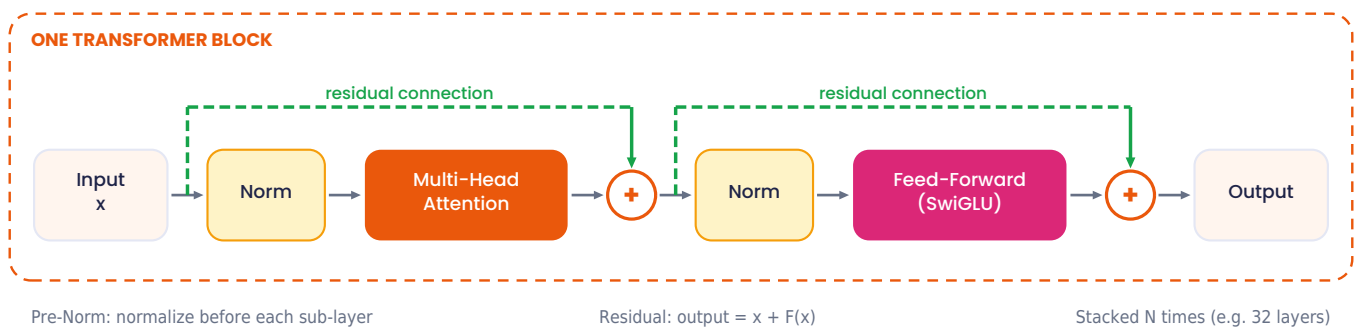
Each block has two sub-layers: **multi-head self-attention** (mixes information across tokens) and a **feed-forward network** (processes each token on its own). Both are wrapped with normalisation and a residual connection, and blocks are stacked N times.

EXPLAINED

- **Residual connections** compute $x + F(x)$, giving gradients a clean path and making very deep stacks trainable.
- **Normalisation** (LayerNorm or RMSNorm) stabilises activations; modern LLMs use **Pre-Norm**, normalising before each sub-layer.
- The **FFN** expands the dimension (about 4x, or $\sim 8/3$ with gated variants), applies GELU or SwiGLU, then projects back. It holds roughly two-thirds of a block's parameters.
- Attention = communication between tokens; FFN = computation and memory inside each token.

DIAGRAM

Inside one Transformer block

**WORKED EXAMPLE**

Llama-2-7B stacks 32 such blocks with $d_{\text{model}} = 4096$, 32 heads and an FFN width of 11,008. Every block has the same shape, only the learned weights differ.

INTERVIEW TIP

Practise drawing it: Norm, Attention, Add, Norm, FFN, Add.

LIKELY FOLLOW-UP

Why is the FFN so large? It is often viewed as the block's key-value memory for facts and patterns, and it holds about two-thirds of the block's parameters.

QUESTION 09 / 15

Compare encoder-only, decoder-only and encoder-decoder models.

SHORT ANSWER

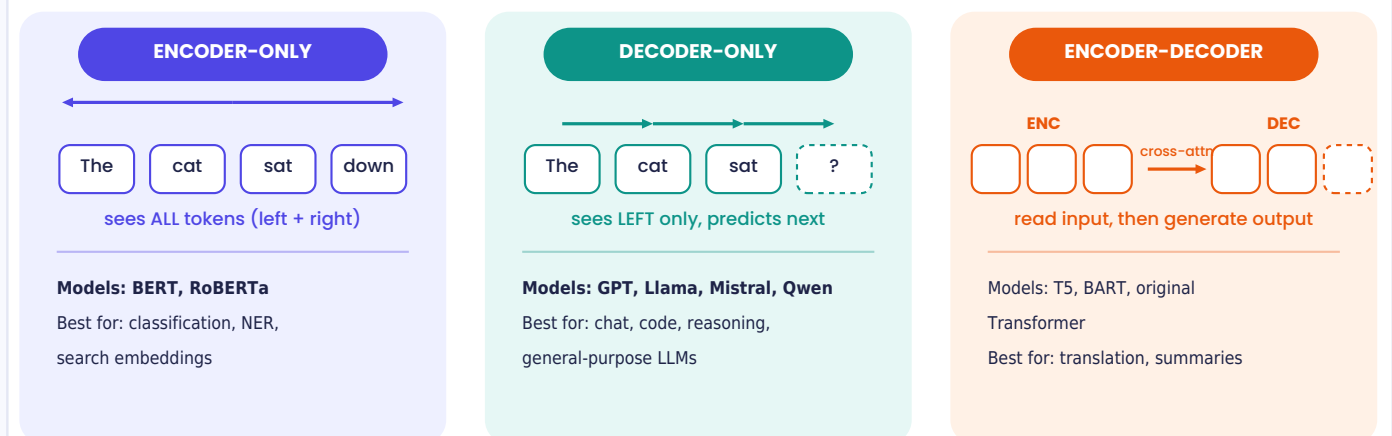
Encoder-only models read the whole text in both directions to understand it. **Decoder-only** models read left to right and generate. **Encoder-decoder** models encode an input, then decode an output while using cross-attention to look back at it.

EXPLAINED

- **Encoder-only** (BERT, RoBERTa): trained with masked language modelling; best for classification, NER and embeddings.
- **Decoder-only** (GPT, Llama, Mistral, Qwen): trained to predict the next token; dominant for chat, code and reasoning.
- **Encoder-decoder** (T5, BART, original Transformer): strong for translation and summarisation with a clear input/output split.
- Why decoder-only dominates: one simple objective, scales well, and every task becomes text-in, text-out through prompting.

DIAGRAM

Three Transformer families



WORKED EXAMPLE

Input: "**Great battery life!**". BERT outputs a label (positive). A GPT-style model continues the text: "Sentiment: Positive". T5 can be asked: "translate English to German: Great battery life!"

INTERVIEW TIP

Explain why decoder-only won: one objective, easy scaling, and prompt-based flexibility.

LIKELY FOLLOW-UP

Can decoder-only models do classification? Yes: frame it as a prompt and read the generated label, or put a small classification head on the final hidden state.

QUESTION 10 / 15

What is causal masking, and what does autoregressive generation mean?

SHORT ANSWER

In decoder-only models a **causal mask** stops each token from attending to future tokens. This makes training match generation: the model only ever uses the **past** to predict what comes next.

EXPLAINED

- The mask sets future positions to **-infinity** before softmax, so their attention weights become exactly 0.
- **Training**: every position is predicted in parallel in one pass (teacher forcing); each row of the mask is a separate prediction task.
- **Inference**: tokens come out one by one because token $t+1$ depends on token t . Prompt processing ("prefill") is parallel; generation ("decode") is sequential.

DIAGRAM

Causal attention mask

query	The	cat	sat	on	mat	key
The	✓	−∞	−∞	−∞	−∞	
cat	✓	✓	−∞	−∞	−∞	
sat	✓	✓	✓	−∞	−∞	
on	✓	✓	✓	✓	−∞	
mat	✓	✓	✓	✓	✓	

HOW TO READ THE MASK

- Row = the token asking (query)
- Column = the token it may read (key)
- Green = allowed; grey = masked with $-\infty$
- $\text{softmax}(-\infty) = 0$, so the future gets zero weight
- Outlined row: predicting after "sat"
- sees only: The, cat, sat

Training: all rows run in parallel

Generation: one new row per step

WORKED EXAMPLE

For "**The cat sat on mat**", when predicting the word after "sat" the model sees only "The cat sat". It cannot peek at "on mat", otherwise it could simply copy the answer during training.

INTERVIEW TIP

Separate prefill (parallel) from decode (sequential); it shows systems-level awareness.

LIKELY FOLLOW-UP

What is speculative decoding? A small draft model proposes several tokens and the large model verifies them in one parallel pass, speeding up generation without changing the output distribution.

QUESTION 11 / 15

How do greedy, temperature, top-k and top-p decoding work?

SHORT ANSWER

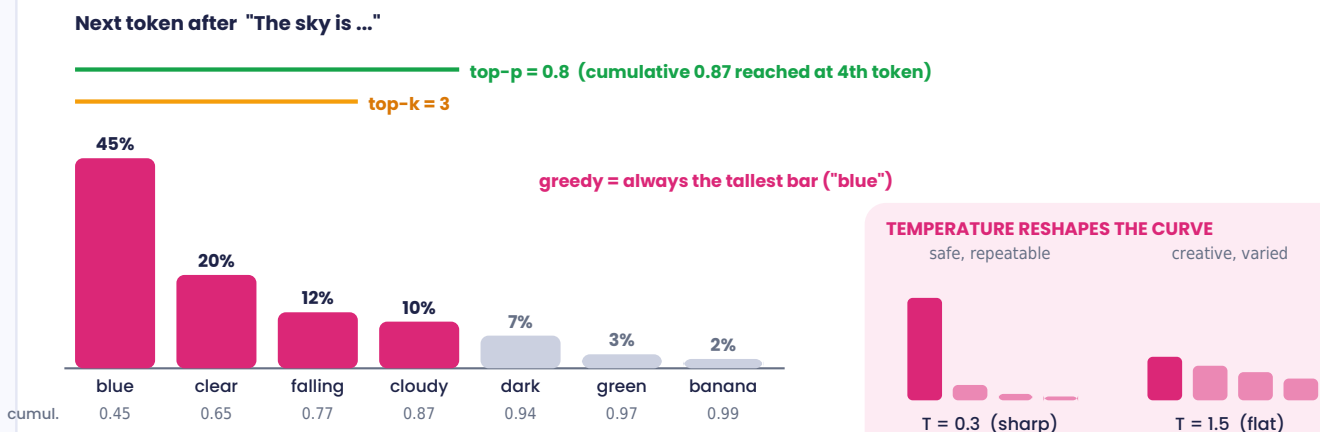
The model outputs probabilities; the **decoding strategy** decides which token to pick. Greedy is deterministic, while sampling with temperature, top-k and top-p trades accuracy against diversity.

EXPLAINED

- **Greedy**: always take the most likely token. Deterministic but can be repetitive. **Beam search** keeps several candidate sequences (common in translation).
- **Temperature T**: divide logits by T before softmax. $T < 1$ sharpens (safer), $T > 1$ flattens (more creative); T near 0 approaches greedy.
- **Top-k**: sample only from the k most likely tokens. **Top-p (nucleus)**: sample from the smallest set whose cumulative probability reaches p; it adapts to model confidence.
- Rule of thumb: low temperature for code, extraction and factual Q&A; higher for brainstorming and creative writing.

DIAGRAM

Probability bars and sampling cut-offs



WORKED EXAMPLE

For "The sky is ____" with probabilities blue 45%, clear 20%, falling 12%, cloudy 10%: top-k = 3 keeps {blue, clear, falling}; top-p = 0.8 keeps {blue, clear, falling, cloudy} because the cumulative total first passes 0.8 at 0.87.

INTERVIEW TIP

Link each setting to a use case; interviewers want to hear when you would change it.

LIKELY FOLLOW-UP

What does temperature 0 give you? Near-deterministic greedy output, although floating-point effects and batching can still cause small variations.

QUESTION 12 / 15

What is the context window, and how does the KV cache speed up inference?

SHORT ANSWER

The **context window** is the maximum number of tokens (prompt + output) the model can attend to at once. The **KV cache** stores the Keys and Values of past tokens so each new token only computes its own, avoiding repeated work.

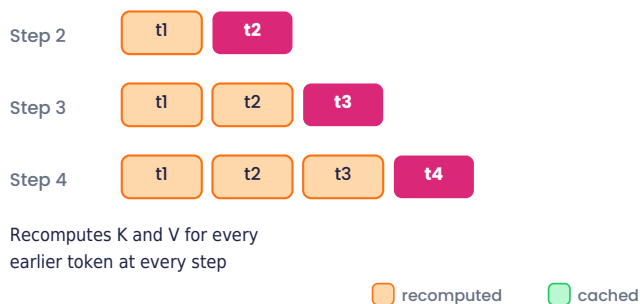
EXPLAINED

- **Without a cache:** generating token n recomputes K and V for all $n-1$ earlier tokens at every step.
- **With a cache:** K and V are computed once per token, stored, and reused, so each step handles only the new token.
- **Cost:** cache memory grows linearly with sequence length, layers and batch size, often the main bottleneck for long-context serving.
- **Remedies:** GQA / MQA, KV quantisation, paged attention (vLLM), sliding windows and prefix caching.
- **KV bytes** = $2 \times \text{layers} \times \text{kv_heads} \times \text{head_dim} \times \text{seq_len} \times \text{bytes/value}$

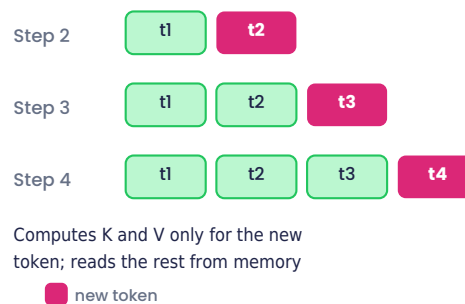
DIAGRAM

Recompute everything vs reuse the cache

WITHOUT KV CACHE



WITH KV CACHE



WORKED EXAMPLE

A Llama-2-7B-style model (32 layers, 32 KV heads, head_dim 128, fp16) at 4,096 tokens: $2 \times 32 \times 32 \times 128 \times 4096 \times 2 \text{ bytes} = \text{about } 2 \text{ GiB per sequence}$.

INTERVIEW TIP

Quote the memory formula, then name GQA and paged attention as mitigations.

LIKELY FOLLOW-UP

What is prefix caching? Reusing the KV cache of a repeated prompt prefix (such as a system prompt) across requests to save compute and latency.

QUESTION 13 / 15

Explain the LLM training pipeline: pre-training, SFT and RLHF / DPO.

SHORT ANSWER

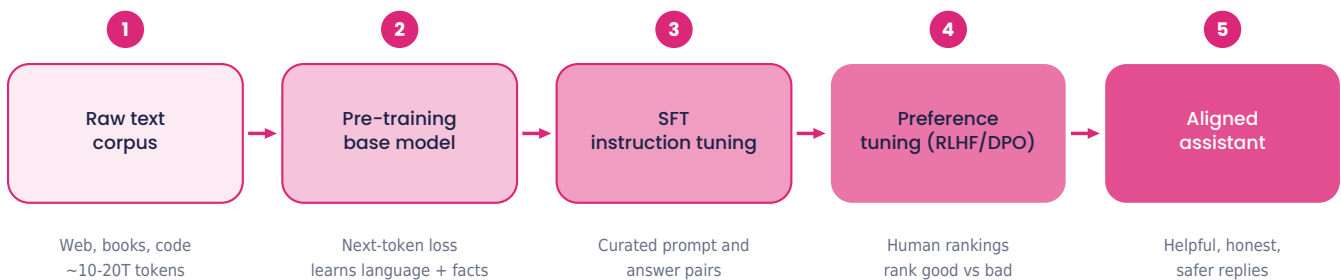
Modern LLMs are built in stages. **Pre-training** on massive raw text teaches language and world knowledge; **supervised fine-tuning (SFT)** teaches instruction following; **preference tuning (RLHF or DPO)** aligns answers with human preferences for helpfulness and safety.

EXPLAINED

- **Pre-training**: next-token prediction (cross-entropy loss) on trillions of tokens; by far the most expensive stage.
- **SFT**: train on curated (prompt, ideal response) pairs, turning a base model into an assistant.
- **RLHF**: learn a reward model from human rankings, then optimise the policy (e.g. PPO) with a KL penalty. **DPO** skips the reward model and optimises directly on preference pairs. RL with verifiable rewards is also used to improve reasoning in math and code.
- **LoRA** and similar methods fine-tune small low-rank adapters instead of all weights.

DIAGRAM

From raw text to assistant



Most compute is spent in stage 2; stages 3 and 4 use far less data but shape behaviour

WORKED EXAMPLE

A base model asked "How do I boil an egg?" may continue with more questions. After SFT and preference tuning it replies with clear, polite, step-by-step instructions.

INTERVIEW TIP

State the purpose of each stage: knowledge, instruction following, preference alignment.

LIKELY FOLLOW-UP

RLHF or DPO: which one? DPO is simpler and cheaper (no reward model or RL loop); PPO-style RLHF offers more flexibility with online feedback.

QUESTION 14 / 15

What are scaling laws, and what did Chinchilla teach us?

SHORT ANSWER

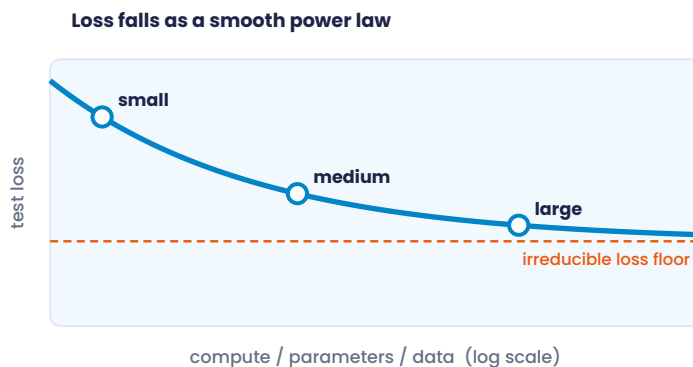
Scaling laws show that LLM loss falls **predictably as a power law** with more parameters, data and compute. Chinchilla (2022) showed that for a fixed compute budget, model size and training tokens should grow together: roughly **20 tokens per parameter**.

EXPLAINED

- Kaplan et al. (2020) reported smooth power-law trends. Hoffmann et al. (2022, Chinchilla) found earlier models were **under-trained**: too many parameters, too little data.
- **Chinchilla 70B** (1.4T tokens) outperformed **Gopher 280B** (300B tokens) while using the same training compute.
- Today teams often **over-train** smaller models far beyond 20 tokens per parameter, because inference cost over a model's lifetime outweighs training cost.
- Scaling laws guide budgeting and forecasting, but benchmark gains and "emergent" abilities do not always follow smooth curves.

DIAGRAM

Loss vs scale, and the Chinchilla result



Gopher (2021)
280B params | 300B tokens

Chinchilla (2022)
70B params | 1.4T tokens
same compute, better results

Rule of thumb
 $\text{tokens} \approx 20 \times \text{parameters}$

WORKED EXAMPLE

Llama 3 8B was trained on about **15T tokens, roughly 1,900 tokens per parameter**, about 90x beyond the compute-optimal 20 (which would be only 160B tokens). The result is a small model that is cheap to serve.

INTERVIEW TIP

Explain the training-vs-inference cost argument for over-training small models.

LIKELY FOLLOW-UP

Do scaling laws still hold in 2026? Pre-training loss still follows smooth trends, but labs now combine it with better data, post-training and test-time compute.

QUESTION 15 / 15

What is Mixture of Experts (MoE), and why is it used?

SHORT ANSWER

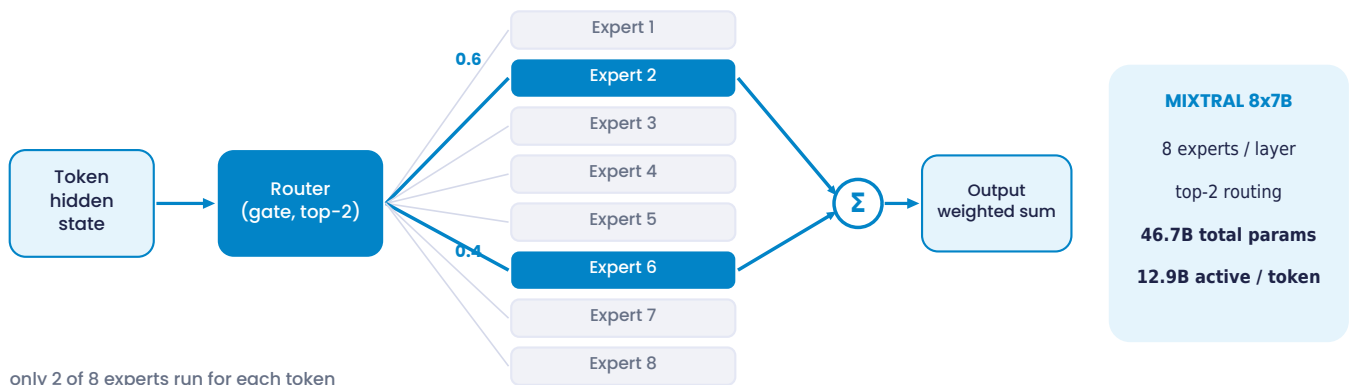
MoE replaces the dense feed-forward layer with **many parallel "expert" FFNs** plus a small **router** that sends each token to only the top-k experts. The model has huge total capacity but uses only a fraction of its parameters for each token.

EXPLAINED

- The **router (gating network)** scores experts per token, selects the top-k (commonly 2) and combines their outputs using the gate weights.
- **Benefit:** more knowledge capacity at roughly the compute cost of a smaller dense model, so cheaper FLOPs per token.
- **Challenges:** load balancing (avoid starving experts), all experts must still fit in memory, and communication overhead in distributed training.
- Attention layers usually stay dense and shared; only the FFN part is sparse.

DIAGRAM

Router sends each token to top-2 experts



WORKED EXAMPLE

Mixtral 8x7B has 8 experts per layer and uses top-2 routing: about **46.7B total parameters** but only **~12.9B active** per token, giving the quality of a much larger model at roughly the speed of a ~13B dense one.

INTERVIEW TIP

Distinguish total vs active parameters; it is the most common MoE follow-up question.

LIKELY FOLLOW-UP

How is MoE different from an ensemble? An ensemble runs several full models; MoE routes tokens inside one model, and the experts are trained jointly with the router.

QUICK REVISION

Rapid-Fire Revision Sheet

Read this the night before your interview: one line per question.

#	TOPIC	ONE-LINE ANSWER
01	What is an LLM and how does it generate text?	LLM = Transformer trained to predict the next token; generation repeats that step.
02	Tokenization and subword BPE	BPE subwords balance vocabulary size and sequence length; cost is counted in tokens.
03	Embeddings and vector meaning	Embeddings map token IDs to dense vectors where distance reflects meaning.
04	Transformers vs RNNs / LSTMs	Attention gives O(1) paths between tokens and parallel training; RNNs are sequential.
05	Self-attention: Query, Key, Value	$\text{softmax}(QK^T/\sqrt{d})V$: queries match keys, the weights mix the values.
06	Multi-head attention (and GQA)	Several heads learn different relations in parallel; GQA shares K/V to save memory.
07	Positional encoding and RoPE	Attention is order-blind, so positions are injected; RoPE encodes relative distance.
08	Anatomy of a Transformer block	Attention + FFN, each with a norm and a residual connection, stacked N times.
09	Encoder-only vs decoder-only vs enc-dec	Encoder = understand, decoder = generate, encoder-decoder = map input to output.
10	Causal masking and autoregression	The causal mask hides the future (-inf, weight 0); training is parallel, decoding sequential.
11	Decoding: greedy, temperature, top-k, top-p	Temperature, top-k and top-p control randomness; greedy is deterministic.
12	Context window and KV cache	KV cache stores past Keys/Values to skip recomputation, at the cost of growing memory.
13	Training pipeline: pre-train, SFT, RLHF/DPO	Pre-train (knowledge), then SFT (instructions), then RLHF / DPO (preferences).
14	Scaling laws and Chinchilla	Loss follows power laws; Chinchilla: ~20 tokens per parameter, today often over-trained.
15	Mixture of Experts (MoE)	MoE routes each token to top-k experts: large capacity, small active compute.

Found this useful? Follow for more AI & LLM interview prep on **LinkedIn: @Zakaurrehman**